THOMPSON RIVERS UNIVERSITY · DEPARTMENT OF COMPUTING SCIENCE

COMP 1110 — Introduction to Computer Programming (Python)

Course outline, Fall 2026

- **Instructor**: Dr. Quan Nguyen
- **Email**: lnguyen[at]tru[dot]ca
- **Office Hours**: Thursdays, 11:30 AM – 12:30 PM — Clock Tower CT 409

Course Description

COMP 1110 is a first course in computer programming. No prior programming experience is assumed. Using Python, students learn how a computer executes instructions and how to turn a problem statement into working, tested code. The course moves from the basic building blocks — variables, input/output, conditionals, and loops — through functions and modular design, into Python's core data structures (strings, lists, dictionaries, tuples, sets) and working with files and real data.

Textbook

Required (free): Charles R. Severance, *Python for Everybody: Exploring Data Using Python 3*.

- **Read it online: https://www.py4e.com/html3/**

Course Learning Outcomes

By the end of this course, students will be able to:

- **CLO 1 — Apply fundamental programming concepts.** Write Python programs using variables, input/output operations, conditional statements, and loops to solve structured computational problems.
- **CLO 2 — Design modular programs using functions.** Design and implement Python programs that organize logic into reusable functions using parameters and return values.
- **CLO 3 — Use data structures to store and manipulate information.** Develop Python programs that use lists, tuples, sets, and dictionaries to organize, process, and analyze collections of data.
- **CLO 4 — Debug and test programs.** Identify, analyze, and correct syntax, runtime, and logical errors in Python programs using systematic debugging and testing strategies.
- **CLO 5 — Apply programming to solve simple problems.** Design and implement Python programs that solve introductory real-world or computational problems using appropriate programming constructs and algorithmic thinking strategies.

Assessment Overview

Component	Weight	Format	When
Weekly Worksheets (10 × 2%)	**20%**	Individual, on the COMP 1110 Practice Platform (link posted on Moodle)	Due every Thursday, 11:59 PM
Midterm 1	**25%**	Closed-book, 60 min, on Moodle in class	Thu, Oct 15 (Week 6)
Midterm 2	**25%**	Closed-book, 60 min, on Moodle in class	Thu, Nov 19 (Week 11)
Final Project	**25%**	Group build, **individually graded**	Proposal Nov 5 · Presentations Dec 3 & 8 · Final submission Dec 8
Attendance & Participation	**5%**	Moodle QR check-in each class	Every class
Total	**100%**		

Weekly Worksheets — 20%

- Access the COMP 1110 TRU-Think platform (link posted on Moodle)
- Ten worksheets, 2% each. Each worksheet contains short programming exercises drawn from the week's textbook chapter plus one *trace-and-explain* question (predict the output, or find and fix the bug).
- Worksheets are graded as Pass/Fail. To receive a Pass, you must attempt all questions and correctly solve at least 80% of them

Midterms — 25% each

- Two **closed-book, 60-minute** assessments written **on Moodle during class time**.
- **Closed-book means:** no textbook, no notes, no cheat sheet, no AI assistants, no browsing beyond the Moodle quiz page, no communication with anyone.
- You may **not** run Python during the exam — you must be able to read code without executing it, which is exactly what the weekly trace-and-explain questions prepare you for.

	Coverage
Midterm 1 (Thu Oct 15, 9:30–10:30)	Chapters 1–5: variables, expressions, conditionals, functions, iteration
Midterm 2 (Thu Nov 19, 9:30–10:30)	Chapters 6–10: strings and string formatting, files, lists, dictionaries, tuples, sets · testing and docstrings

Midterm 2 is *cumulative in tools* — you will still need conditionals, loops, and functions — but its new material is drawn from Chapters 6–10.

Final Project — 25%

Overview & Requirements

- **Teams:** 3–4 students. Teams are formed by Week 3 (Sept 24) and will also serve as your study group for the term.
- **Scope:** 400–600 lines of Python that reads data, processes it, and produces a useful result.

- **Code Standards:** Submissions must include at least one custom class, automated tests (`assert` or `unittest`), docstrings for every function, and PEP 8 formatting.

GitHub (Mandatory)

Every team must keep its code in a **GitHub repository**, and every member must **commit and push their own contributions under their own GitHub account**. Do not have one person push the whole project on everyone's behalf, and do not paste teammates' code into your own commits.

The commit history is the evidence of your contribution. It is what I check when I corroborate peer assessments, and a member whose contribution does not appear in the history has no record of having contributed — whatever the team says afterwards. Commit as you go, in reasonably sized pieces with meaningful messages, rather than in a single push at the end.

The repository must be **public**, so that I can review the code and the commit history without being added to it. Include the repository URL in your project proposal and your final submission. Because every team's repository is public, you will be able to read other teams' code: you may not copy from it. See Academic Integrity.

How the individual grade is computed

Component	Weight (of final grade)	Basis
Individual presentation & Q&A	**12%**	Your own 4-minute segment of the team presentation, plus your answers when questioned about the code
Team deliverable (code, tests, documentation), **adjusted by peer factor**	**10%**	One team mark, multiplied by your individual peer-assessment factor
Project proposal (team)	**3%**	1-page team submission: problem, inputs, outputs, planned functions, who owns what, GitHub repository URL

Course Schedule

Classes run Tuesdays and Thursdays, 9:30 – 11:20 AM in OM 1241. Chapter numbers refer to *Python for Everybody*.

Week	Dates	Topic	Reading	Due
1	**Thu Sep 10**	Course intro · What is a program? · Hardware, interpreters, and your first `print` · Setting up Python + VS Code	Ch. 1 — Introduction	—
2	Tue Sep 15 Thu Sep 17	Values, types, variables, expressions · `input()`, type conversion, operators, mnemonic names, comments	Ch. 2 — Variables	—
3	Tue Sep 22 Thu Sep 24	Boolean expressions and logical operators · `if` / `elif` / `else`, nested conditionals · `try` / `except` · **Project teams formed (Thu)**	Ch. 3 — Conditionals	**W1** (Mon Sep 21) **W2** (Thu Sep 24) **Teams due** (Thu)

Week	Dates	Topic	Reading	Due
4	Tue Sep 29 Thu Oct 1	Functions: definition, invocation, parameters, arguments, return values · Scope · Why we decompose	Ch. 4 — Functions	**W3** (Thu Oct 1)
5	Tue Oct 6 Thu Oct 8	Iteration: `while`, `for`, `break`, `continue` · Loop patterns: counting, summing, max/min · Loop-and-a-half	Ch. 5 — Iterations	**W4** (Thu Oct 8)
6	Tue Oct 13 **Thu Oct 15**	**Introduction to Git & GitHub:** repositories, cloning, `commit` / `push`, reading a commit history, working on a repository as a team · **Final project deliverables explained:** proposal, team repository, presentation & Q&A, peer assessment, and how the individual grade is computed (Tue) **MIDTERM 1** — Moodle, closed-book, 9:30–10:30 (Thu)	GitHub — Hello World Review Ch. 1–5 for Thursday	**W5** (Fri Oct 16) **Midterm 1**
7	Tue Oct 20 Thu Oct 22	Strings as sequences · Indexing, slicing, immutability · Traversal, `in`, string methods · **String & output formatting:** f-strings, `format()`, alignment, rounding	Ch. 6 — Strings	—
8	Tue Oct 27 Thu Oct 29	Files: `open`, reading line by line, searching, writing files · Handling missing files (Tue) **Testing & documentation:** test plans, edge cases, `assert`, intro to `unittest`, docstrings, README files (Thu)	Ch. 7 — Files A Beginner's Guide to Unit Tests in Python Documenting Python Code: A Complete Guide	**W6** (Thu Oct 29)
9	Tue Nov 3 Thu Nov 5	Lists: mutability, traversal, slices, methods, `split` / `join` · Aliasing and list arguments · Parsing lines	Ch. 8 — Lists	**W7** (Thu Nov 5) **Team proposal** (Thu)
10	**Tue Nov 10** ~~Thu Nov 12~~	Dictionaries: key-value structures, counting patterns, `get`, looping over dictionaries *(no class Thu — fall mid-term break)*	Ch. 9 — Dictionaries	**W8** (Thu Nov 12)
11	Tue Nov 17 **Thu Nov 19**	Tuples, sorting with `sorted` and `items()` · Sets and unique collections *(supplementary notes)* · Review (Tue) · **MIDTERM 2** — Moodle, closed-book, 9:30–10:30 (Thu)	Ch. 10 — Tuples + Moodle notes	**W9** (Fri Nov 20) **Midterm 2**
12	Tue Nov 24 Thu Nov 26	**Object-oriented programming I:** why classes, defining a class, attributes, methods, `__init__`, objects vs. dictionaries (Tue) **OOP II:** multiple instances, object lifecycle, a first look at inheritance (Thu)	Ch. 14 — Objects	**W10** (Thu Nov 26)
13	Tue Dec 1 Thu Dec 3	**Formatting & code style:** PEP 8, naming, layout, comments that earn their place, auto-formatters and linters · Polishing your project for submission (Tue) **Final project presentations & Q&A** (Thu)	PEP 8 style guide	**W11** (Thu Dec 3)

Week	Dates	Topic	Reading	Due
14	**Tue Dec 8** *(last day of classes)*	**Final project presentations & Q&A** (continued) · Where to go next: databases, web data, and data visualization · Course wrap-up	—	**Presentations Final project due 11:59 PM**

> **Tuesday, December 8 is the last day of classes.** Final code, documentation, and confidential peer assessments are due that night at **11:59 PM**.
>
> **There is no final exam in this course.** The December 10–23 examination period does not apply to COMP 1110 — once you have presented and submitted, you are finished.

Readings beyond the textbook. Four topics in this course are not in *Python for Everybody*, so their readings are linked directly in the schedule above:

Topic	Week	Reading
Git & GitHub	6	GitHub — Hello World (GitHub Docs)
Unit testing	8	A Beginner's Guide to Unit Tests in Python (Dataquest)
Documentation & docstrings	8	Documenting Python Code: A Complete Guide (Real Python)
Code style	13	PEP 8 — Style Guide for Python Code

Read the testing and documentation articles for their **`assert`**, **`unittest.TestCase`**, **and docstring** sections. Both go further than this course requires — you are not responsible for `setUp` / `tearDown`, Sphinx, or doctest.

Note on sets: *Python for Everybody* does not devote a chapter to sets. Supplementary notes and exercises covering sets and set operations will be posted on Moodle for Week 11 (CLO 3).

Communication

- **Course-related questions** (syntax, errors, worksheet clarifications): use the **Moodle discussion forum**, so that other students benefit from the answer. Post your code and the full error message — not a screenshot description of it.
- **Individual matters** (academic concessions, extensions, accessibility): email **lnguyen[at]tru[dot]ca**.
- **Response time:** within 24 hours during normal working hours (9 AM – 5 PM, Mon–Fri).

Course Policies

Attendance

A registered student who does not attend the first two events (e.g., lectures/labs) of their course(s) and who has not made prior arrangements acceptable to the instructor(s) may, at the discretion of the instructor(s), be considered to have withdrawn from the course(s) and have their course registration(s) deleted.

- Attendance accounts for **5%** of your final grade and is taken via **Moodle QR code** in class.
- Arriving more than 5 minutes late will be recorded as absent.
- Missing more than 30% of class sessions will result in automatic failure of the course.
- Each student gets **three "free passes"** for any reason (illness, family matters, commuting issues) without penalty — no documentation needed, no email required.

Please refer to TRU's attendance policy.

Academic Concessions

If circumstances (e.g., illness, family emergency, significant life event) may prevent you from meeting course requirements:

- Notify the instructor **at least 24 hours before the deadline**.
- Requests are considered **case-by-case**; you may be asked for documentation.
- Possible accommodations: deadline extensions, alternative assessments, or a deferred midterm.
- Requests made **after the deadline** are usually refused.

Late Work

- **Worksheets:** accepted up to 48 hours late at **−50%**; not accepted after that. Because the lowest worksheet score is dropped, one missed worksheet costs you nothing.
- **Final project deliverables: −25% per day**, up to −75%. After 3 days late, work is not accepted (grade = 0). Because December 8 is the last day of classes and final grades must be submitted shortly after, **no extension beyond December 11 is possible** for any reason short of a formal academic concession granted by the university.
- **Presentations:** cannot be made up. If you miss your presentation slot without an approved concession, that 12% is lost regardless of your team's performance.
- **Midterms:** the Moodle quiz opens and closes at fixed times and must be written at the scheduled time. Deferred midterms are arranged only through the academic concession process above.

Missed Midterms

If you miss a midterm with an approved concession, the standard remedy is a deferred sitting within one week, on a different version of the quiz. If a deferred sitting is not feasible, the weight may be transferred to the other midterm at the instructor's discretion.

Accessibility

Students registered with Accessibility Services who require accommodations must provide their Letter of Accommodation to the instructor as soon as possible. This letter will outline the necessary accommodations to ensure an equitable learning environment. Please arrange this early in the term to allow time for exam and assessment adjustments.

Academic Integrity

Programming courses have a specific hazard: it is very easy to submit code you did not write and do not understand, and very hard to pass the midterms if you have been doing that all term.

- **Worksheets:** discussing approaches with classmates is encouraged. Writing code together, or copying another student's code, is not. Write your own solution, from your own keyboard.

- **Midterms:** entirely individual, no collaboration, no outside help.
- **Final project:** collaboration *within* your team is the point; collaboration *between* teams is not. Project repositories are public, which means you can read other teams' code — you may not copy from it. Taking code from another team's repository is plagiarism, and it is easy to detect: both repositories are timestamped and I read the commit histories. External code (tutorials, Stack Overflow, library documentation) must be cited in your README with a comment in the code showing what came from where.

Cases of academic dishonesty are reported per TRU policy.

Policy on the Use of Generative AI

I am a proponent of the **responsible** and **ethical** use of AI in education. AI tools are now part of how professional programmers work, and pretending otherwise would not serve you. However, in an introductory course the point of every exercise is to build the mental model that lets you *read* and *judge* code — and AI can quietly remove exactly that practice.

What is permitted:

- Asking an AI tool to explain a concept, an error message, or a piece of textbook code.
- Asking for extra practice problems, or for a hint when you are stuck.
- Using AI to review code you have already written yourself, and to explain its suggestions.

What is not permitted:

- Submitting AI-generated code as a worksheet or project solution that you cannot explain line by line.
- Any AI use during midterms.

A practical test: if I ask you in class to explain any line of your submitted code and change it on the spot, you should be able to. Worksheet and project marks may be adjusted following such a conversation. Note that the final project's Q&A component (12% of your final grade) is precisely this test, conducted formally — and the midterms are closed-book with no AI access, so code you never understood is a debt that comes due twice.

For worksheets, the **AI tutor built into the Practice Platform** is the sanctioned AI help: it responds to every submission with Socratic feedback — questions and hints aimed at getting you to the answer yourself, rather than the answer. Use it. It is designed for this course, it knows which worksheet you are on, and it will not hand you code you cannot explain.

Please refer to TRU's guideline on the use of generative AI for more information.

How to Succeed in This Course

1. **Type the code, don't read it.** Run every example from the textbook yourself. Change a value, break it deliberately, see what error you get.
2. **Start worksheets as soon as they are released.** You will get stuck — that is the point — and you need the days before the deadline to be unstuck.
3. **Read error messages from the bottom up.** The last line names the error; the line above it names the line number.
4. **Ask early.** A five-minute question on Moodle on Monday saves a frustrating Wednesday night.

5. **Keep every program you write.** By Week 12 your own past code is your best reference.